Temporal Multiple Rotation Averaging on a Distributed Dynamic Network

Aidan Blair, Amirali Khodadadian Gostar, Ruwan Tennakoon, Alireza Bab-Hadiashar, *Senior Member, IEEE*, and Reza Hoseinnezhad

***Abstract*—This article proposes a solution for multiple rotation averaging on time-series data such as video. In applications using video data such as target tracking, in addition to the data found in individual frames, temporal information across multiple frames such as target trajectories can be used to more accurately estimate target states. Existing techniques for robust rotation averaging, including traditional iterative optimization and emerging neural network methods, do not exploit this temporal information. We first introduce the problem of using temporal data in rotation averaging and propose an extension to existing multiple rotation averaging methods via temporal rrotations. We then propose implementing a motion model for the cameras and predicting camera states using a particle filter, which are used to initialize the rotation averaging algorithm. These methods' performance is evaluated through a Monte Carlo Simulation on synthetic data and compared to an existing method. The results show that using temporal data in time-series datasets significantly increases the accuracy compared to the traditional algorithm for rotation averaging.**

***Index Terms*—Multiple rotation averaging, structure from motion, 3D rotation, temporal information, particle filter.**

I. Introduction

With the advancement of low cost, high-performance vision sensors and the increasing adoption of IoT devices, distributed networks of cameras are becoming increasingly common in many surveillance applications. These applications include but are not limited to sensor control [1], information fusion [2], multi-view reconstruction [3] and multi-object tracking [4]. There are many real-world scenarios utilizing these applications, including networks of connected vehicles, flocks of UAVs, pan-tilt-zoom camera networks and defence applications. In these scenarios, accurate knowledge of camera parameters is crucial to ensure satisfactory performance is achieved. There are two types of camera parameters, intrinsic and extrinsic. Intrinsic parameters are fixed and specific to the camera, these include the focal length, skew factor, offsets and aspect ratio of the camera. Extrinsic parameters are however related to the position of the camera in the world and the heading direction. They change as the camera moves and need to be estimated as part of the sensor registration module of the overall intended task of the system.

A significant problem with existing multi-object tracking methods is that they commonly assume a stationary and static network of cameras in which the state of the cameras are known (i.e. each camera's field of view (FoV) is assumed to be known and stationary). This is not reflective of many real-world applications such as connected vehicles, flocks of UAVs or pan-tilt-zoom camera networks where the network has dynamic and time-variable structure, i.e. the position and orientation of each camera can change and therefore cannot be assumed as prior information. Therefore, the FoVs need to be estimated for accurate information fusion, sensor control, or multi-view reconstruction in a dynamic network. This article focuses only on estimating the orientation of each camera FoV in a large camera network.

There are several existing approaches for estimating camera orientation. Some methods consider using either a combined IMU and GPS or dual-GPS sensors to accurately calculate orientation [5]. However, these methods require expensive sensors and fail when a satellite signal cannot be established (e.g. in a tunnel or in remote locations). An alternative method using cameras is visual odometry, which is commonly used for localization in UAVs [6], however with high speeds and obscurity of scale [7] visual odometry may not yield reliable results.

Multiple Rotation Averaging (MRA) is a method of estimating camera FoVs using the visual information available from the cameras in the network. MRA uses the estimated relative orientations of the cameras to ascertain the absolute camera FoVs. In other words, MRA is the problem of determining the set of absolute camera rotations given a set of noisy relative camera rotations. Rotation averaging has been used in Structure-from-Motion (SfM) problems for decades, and multiple rotation averaging can outperform alternative methods in tasks such as image reconstruction [8] and absolute camera orientation estimation [9]. This article reformulates the Multiple Rotation Averaging (MRA) algorithm for a dynamic and time-varying camera sensor network.

Additionally, existing methods have been proposed for discrete sets of images, equivalent to a single frame of a video. Therefore while they can be applied to video data by simply treating each time step as a discrete problem, they do not consider the temporal information that a dynamic camera network provides. This temporal information, including the motion of the cameras

Manuscript received 3 May 2022; revised 22 July 2023; accepted 24 August 2023. Date of publication 11 September 2023; date of current version 17 October 2023. This work was supported by the Australian Research Council under Grant DE210101181. The associate editor coordinating the review of this manuscript and approving it for publication was Dr. Yongqiang Wang. *(Corresponding author: Amirali Khodadadian Gostar.)*

The authors are with the RMIT University, Melbourne, VIC 3000, Australia (e-mail: s3601670@student.rmit.edu.au; amirali.khodadadian@rmit.edu.au; ruwan.tennakoon@rmit.edu.au; abh@rmit.edu.au; reza.hoseinnezhad@rmit.edu.au).

Digital Object Identifier 10.1109/TSIPN.2023.3313817

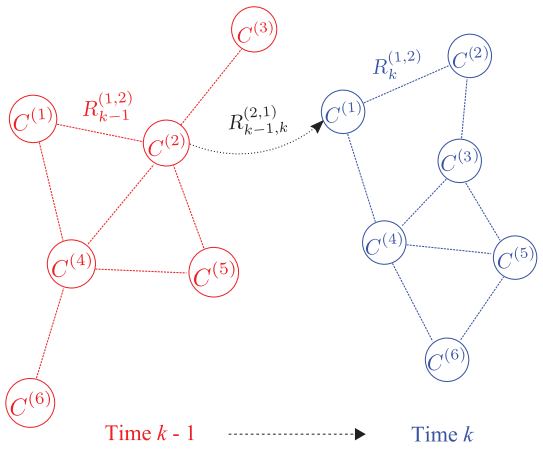


Fig. 1. Graph representation of a camera network at consecutive time steps, $k-1$ (shown in red) and k (shown in blue). Each node represents a camera, and a line between two nodes indicates the presence of a measured relative rotation (e.g. $R_k^{(1,2)}$). An example relative rotation between a node from time step $k-1$ and a node from time step k is shown in black, $R_{k-1,k}^{(2,1)}$.

and the vehicles they are attached to, can be vital to include in challenging scenarios. In a scenario where a flock of UAVs are detecting and tracking multiple dynamic targets in a large area, individual UAVs may frequently dropout from communication with the rest of the flock due to a number of reasons, including moving outside of communication range, obstacles or terrain blocking the communication signal and signal interference. In this situation the ability to use the available temporal information to continue accurately estimating camera orientations through dropouts is crucial.

Fig. 1 shows a simple example of a camera network that demonstrates the dynamic nature of node positions and connections through time. Fig. 1 also reveals the potential temporal information that can be used for MRA. We propose to include the relative rotations between connected cameras from the previous time step to the current time step to increase the accuracy of the motion averaging algorithm. In addition, we propose incorporating a local motion model for each camera in the network and using it within a particle filter to improve initialization accuracy for the following time step.

To demonstrate the accuracy improvements of rotation averaging by using the available temporal information in time-series data, we first construct synthetic datasets for testing. Then baseline results are generated by applying a standard MRA algorithm to the datasets. Finally, we reformulate the MRA algorithm to include temporal information and compare the new results to the baseline to show the extent of potential improvements in accuracy that could be achieved.

This article's contributions are as follows:

- introducing the idea of temporal MRA using relative rotations between discrete time steps;
- defining temporal relative rotations and presenting an algorithm using them for MRA which is described in detail for reproduction;
- Presenting an algorithm that uses a particle filter to exploit motion information in addition to relative rotation measurements for MRA.

The rest of this article is organized as follows. In Section II, we provide an overview of the recent literature. Section III provides a background into the mathematical formulation of the MRA problem and its standard solution. In Section IV, we introduce the novel model features that include temporal information. In Section V we discuss the properties of the datasets used, list the results and compare the performance of our model with the base models. Section VI concludes the article.

II. Related Work

A. Non-Linear Optimization

Several rotation averaging techniques based on non-linear optimization have emerged in recent years. Most notably, bundle adjustment-based methods have been commonly used [10]. Bundle adjustment produces a 3D structure for the camera network using least-square minimization of the distance between feature points in the projections of each camera. Bundle adjustment converges to the statistically optimal solution; however, it is computationally expensive and requires a relatively accurate initialization. The first attempts at algebraic methods with fast, linear, non-optimal solutions were limited by the number of cameras that could be used and were significantly affected by noise.

Govindu [11] was the first to introduce an MRA technique that quickly iterated to a non-optimal solution that worked for a relatively large number of cameras. Govindu [12] also introduced rotation averaging through averaging in the Lie algebra of the group of 3D rotations $SO(3)$. This solution has the advantages of the intrinsic property of Lie groups, i.e. that averaging values in the Lie algebraic space produces a result that falls within the Lie algebraic space, and has computational advantages.

Some other competing, non-iterative approaches were devised, such as Discrete-Continuous Optimization (DISCO) [13]. DISCO considers all images in an unstructured dataset at once rather than incrementally forming a solution like a bundle adjustment. This makes it much faster and also more robust than incremental bundle adjustment. It comprises two steps, first using discrete belief propagation based on a Markov random field formulation of constraints between cameras, which includes noisy sources of constraint geotags and vanishing points. The second step is a Levenberg-Marquardt nonlinear optimization. This is a hybrid discrete-continuous optimization, which is efficient and parallelizable.

Hartley et al. [14] modified existing iterative techniques to use the Weiszfeld algorithm and find the l_1 mean of multiple rotations. Previous methods used the l_2 average, which is known to be susceptible to outliers, whose presence cannot be avoided in MRA applications. Using the l_1 average, this algorithm was more robust and didn't require manual or automated outlier pruning. This was later generalized to the l_q average, where $1 \leq q < 2$ [15].

Chatterjee & Govindu [16] expanded on their previous work with Lie group averaging by implementing a model with an l_1

loss function that provides the initialization for an iteratively reweighted least-squares (IRLS) approach. This was followed by using a more robust $l_{\frac{1}{2}}$ loss function [8] compare to previous algorithms (e.g. Weiszfeld and DISCO). The proposed robust loss functions produced more accurate results in the presence of outliers.

B. Low-Rank and Sparse Decomposition

A non-iterative approach using 'low-rank and sparse' (LRS) matrix decomposition has recently been introduced to rotation and motion averaging [9], [17], [18]. LRS matrix decomposition is a form of Principle Component Analysis (PCA), which aims to reduce the dimensionality of multivariate data to simplify the problem while preserving as much useful data as possible [19]. LRS matrix decomposition is introduced by Candes et al. [20] where they prove that the principal components can be robustly recovered from a matrix composed of a low-rank component and a sparse component. This technique estimates the absolute rotations without iteration and is intrinsically robust to both outliers and missing connections. Its computational requirement is also reasonable and the reported results show that it outperforms the Weiszfeld method in accuracy and speed.

C. Artificial Neural Networks

Most recently, artificial neural networks [21] have been employed to solve the MRA problem. This began with using a neural network to calculate weights for each relative rotation before performing a weighted average. The result outperforms traditional methods [22]. The first approach entirely using artificial neural networks was introduced with NeuRoRA [23]. Another attempt was to solve the MRA problem with a message passing neural network (MPNN). The graph structure matches the epipolar graph of the cameras, with nodes representing cameras, and edges representing the existing connections between cameras. The main benefit is the high processing speed that can be achieved with the use of GPU parallelization. However, this method has drawbacks, particularly being non-robust to outliers and requiring manual removal of outliers from the dataset.

An improvement to the NeuRoRA network has been presented [24]. MSPRA uses an improved initialization method and is robust to the presence of outliers, however the network architecture remains cumbersome. This issue was subsequently addressed by recent approaches such as HARA [25] where the spanning tree prioritizes nodes with many strong triplet supports to reduce the chance of outliers being included, and RAGO [26] significantly simplifies MRA problem into a single network and iteratively optimizes global camera orientations without needing to set a root node.

III. Notation & Background

Rotations in 3D can be represented in several formats. The formats that are used in this article are:

- the 3×3 orthonormal rotation matrix representation $R \in \mathbb{R}^{3\times3}$, i.e. $R^\top R = I_3$ and $\det(R) = 1$,
- the Euler angle representation (also known as yaw, pitch and roll), which is three successive rotations of $\boldsymbol{\theta} = (\alpha, \beta, \gamma)$ in degrees around specified rotation axes, and
- the axis-angle representation

$$\boldsymbol{v} = \theta\hat{\boldsymbol{v}} = [v_1\ v_2\ v_3]^\top$$

which is a single rotation θ around axis $\hat{\boldsymbol{v}}$.

Importantly, a rotation in any one of these representations can be easily transformed into one of the other representations. The elements of rotation matrix corresponding to Euler angles (α, β, γ) are given by

$$\begin{aligned}
\text{row 1: } R_{11} &= \cos(\beta)\cos(\gamma)\\
R_{12} &= \sin(\alpha)\sin(\beta)\cos(\gamma) - \cos(\alpha)\sin(\gamma)\\
R_{13} &= \cos(\alpha)\sin(\beta)\cos(\gamma) + \sin(\alpha)\sin(\gamma)\\
\text{row 2: } R_{21} &= \cos(\beta)\sin(\gamma)\\
R_{22} &= \sin(\alpha)\sin(\beta)\sin(\gamma) + \cos(\alpha)\cos(\gamma)\\
R_{23} &= \cos(\alpha)\sin(\beta)\sin(\gamma) - \sin(\alpha)\cos(\gamma)\\
\text{row 3: } R_{31} &= -\sin(\beta)\\
R_{32} &= \sin(\alpha)\cos(\beta)\\
R_{33} &= \cos(\alpha)\cos(\beta)
\end{aligned} \tag{1}$$

Given the rotation matrix, Euler angles are given by:

$$\begin{aligned}
\alpha &= \tan^{-1}(R_{32}/R_{33})\\
\beta &= -\sin^{-1}(R_{31})\\
\gamma &= \cos^{-1}\left(R_{11}/\sqrt{1-R_{31}^2}.\right)
\end{aligned} \tag{2}$$

3D rotation matrices belong to the Special Orthogonal group $\mathbf{SO}(3)$, $R \in \mathbf{SO}(3)$, which are related to the equivalent axis-angle representation $\boldsymbol{v}$ through the associated Lie algebra $[\boldsymbol{v}]_\times \in \mathfrak{so}(3)$, where $[\boldsymbol{v}]_\times$ is the skew-symmetric form of $\boldsymbol{v}$

$$\boldsymbol{\Omega} = [\boldsymbol{v}]_\times = \begin{bmatrix} 0 & -v_3 & v_2 \\ v_3 & 0 & -v_1 \\ -v_2 & v_1 & 0 \end{bmatrix}. \tag{3}$$

The transformation between the rotation matrix representation and the axis-angle representation are given by the matrix exponential and matrix logarithm functions,

$$R = \exp(\boldsymbol{\Omega}) \tag{4}$$

$$\boldsymbol{\Omega} = \log(R) \tag{5}$$

which are defined by

$$\exp(A) = I + \sum_{k=1}^{\infty} \frac{A^k}{k!} \tag{6}$$

$$\log(A) = \sum_{k=1}^{\infty} (-1)^{k+1} \frac{(A-I)^k}{k} \tag{7}$$

Further information on Lie groups, Lie algebras and the matrix exponential and logarithm functions can be found here [27].

If the absolute rotation of cameras i and n in a given reference frame are $R^{(i)}$ and $R^{(n)}$, then the relative rotation from n to i is given by

$$R^{(n,i)} = R^{(i)} R^{(n)^{-1}}. \tag{8}$$

Multiple Rotation Averaging (MRA) is a subset of the Structure-from-Motion problem. It aims to find the absolute rotations of a set of cameras given a set of noisy pairwise relative rotations as well as the absolute orientation of one camera (known as the reference camera). The network of cameras can be described by the graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, where the vertices $\mathcal{V}$ represent the cameras and the edges $\mathcal{E}$ represent the relative relationship between those cameras. The number of vertices/cameras is denoted by $|\mathcal{V}| = N$ and the number of edges by $|\mathcal{E}| = M$.

Representing a camera, each node is characterized by its label and the absolute rotation of the camera which is denoted by, $R^{(i)}$ for node $(i) \in \mathcal{V}$. Each edge in the graph corresponds to a communication link between two cameras through which the images are sent and received, and is characterized by a pair comprised of the node labels and a weight that represents the relative rotation between the two camera. The weight associated with the edge $(n, i) \in \mathcal{E}$ is denoted by $R^{(n,i)}$ and is assumed to be calculated via feature matching between overlapping images acquired by the two cameras.

MRA is the task of finding the absolute rotations, $\{R^{(i)}\}_{i=1}^{N}$, of each node in the network, given a set of noisy relative camera orientations $\{\tilde{R}^{(n,i)}\}$ and a reference camera rotation. With no loss of generality, the reference camera index is assumed to be $i = 1$, and its orientation to be $\boldsymbol{\theta}_0^{(1)} = \mathbf{0}$. The most common approach to solving the MRA problem is via finding a solution for the following optimization problem [16]:

$$R^{(i)*} = \underset{\{R^{(i)}\}}{\arg\min} \sum_{(n,i)\in\mathcal{E}} \rho\left(d\left(\tilde{R}^{(n,i)}, R^{(i)} R^{(n)^{-1}}\right)\right) \tag{9}$$

where $\rho(\cdot)$ is a loss function such as the ℓ_1 or ℓ_2 loss functions [8] and $d(\cdot)$ is a distance measure between two rotations, e.g. the geodesic, chordal or quaternion distance [28]. While there can exist up to $C_2^N = N(N-1)/2$ relative rotations in the network, only $M = N - 1$ suitable edges need to be present to fully span the network and allow for solving the optimization problem of (9). In practice, there are usually $|\mathcal{E}| > N - 1$ relative rotations, i.e. some redundant information is at hand to improve accuracy.

The $\mathbf{SO}(3)$ Lie group has been used for representing 3D rotations in practical applications [29]. For further details on the mathematical background of rotations, rotation distance metrics and rotation averaging, see Hartley et al. [28].

IV. Proposed Method

We propose two novel MRA methods that incorporate temporal information: 1) MRA with appended temporal relative rotations and 2) Temporal MRA using a particle filter and camera motion model.

A. MRA With Appended Temporal Relative Rotations

The first proposed method which incorporates temporal information is based on including relative rotations between images in different time steps. Specifically, we propose including the relative rotations between images in the previous time step and images in the current time step, such as the rotation $R_{k-1,k}^{(2,1)}$ shown in Fig. 1. Exploiting the available information in sequential data allows for a significant increase in the number of possible relative rotation measurements, which in turn, results in considerable error reduction after averaging.

In a scenario where each camera has several noisy measurements, using a greater number of measurements by including relative rotations from a previous frame to the current frame is expected to result in a more accurate estimation. These additional relative rotations will be referred to as *temporal relative rotations*. In general, let us denote camera i's orientation at current time k by the rotation matrix $R_k^{(i)}$, camera n's orientation at an arbitrary previous time $k^* < k$ by the rotation matrix $R_{k^*}^{(n)}$, and the relative rotation from camera n at time k^* to camera i at current time k by $R_{k^*,k}^{(n,i)}$.

We propose to incorporate all the temporal relative rotations between neighboring cameras from time step $k-1$ to time step k into the MRA solution. Algorithmically, the graph is appended with the same camera nodes but with absolute rotations in the previous time steps. Thus, the index for camera i in time step $k-1$ is set to $N+i$, to ensure each camera and time step combination has a discrete unique index. The L1 Weiszfeld MRA algorithm proposed by Hartley et al. [14] and described in Algorithm 1 is used on this dataset, with the absolute rotation estimates for the first N indexed cameras taken as the output.

B. Temporal MRA

In practice, camera movements are almost always smooth. This piece of practical information can be incorporated into the rotation estimation procedure. In our proposed method, called *temporal MRA*, this information is incorporated into the estimation process via formulating the process as a recursive particle filter. At its backbone, the filter needs a motion model and a measurement model. In this section, we first present how those models are defined in our application of distributed multi-camera rotation estimation, then present the overall proposed method.

1) Camera Motion & Measurement Models: Let us consider a state vector for each camera i, denoted by its Euler angles and angular velocities,

$$x^{(i)} = \left[\alpha^{(i)}\ \beta^{(i)}\ \gamma^{(i)}\ \dot{\alpha}^{(i)}\ \dot{\beta}^{(i)}\ \dot{\gamma}^{(i)}\right]^\top = \left[\boldsymbol{\theta}^{(i)^\top}\ \dot{\boldsymbol{\theta}}^{(i)^\top}\right]^\top. \tag{10}$$

Assuming that smooth camera motions follow a *constant velocity* model, at each time k, angular velocities remain constant and only perturbed by small noise:

$$\begin{aligned} \dot{\alpha}_k^{(i)} &= \dot{\alpha}_{k-1}^{(i)} + e_{\dot{\alpha}} \\ \dot{\beta}_k^{(i)} &= \dot{\beta}_{k-1}^{(i)} + e_{\dot{\beta}} \\ \dot{\gamma}_k^{(i)} &= \dot{\gamma}_{k-1}^{(i)} + e_{\dot{\gamma}} \end{aligned} \tag{11}$$

Algorithm 1: L1 Weiszfeld MRA Algorithm Running Locally in Camera i of a Distributed Camera Network.

function L1W_MRA$\boldsymbol{\theta}_0^{(i)}, \mathcal{E}_i, \{R^{(n,i)}\}_{(n,i)\in\mathcal{E}_i}, t_{\max}$
▷ $\boldsymbol{\theta}_0^{(i)}$: Initial Euler angles.
▷ $\mathcal{E}_i$: Graph edges connected to node i
▷ $R^{(n,i)}$: Rotation matrix measured from camera n to i.
▷ $t_{\max}$: No. of iterations sufficient for convergence.
$R_0^{(i)} \leftarrow$ Eul2Rotm$(\boldsymbol{\theta}_0^{(i)})$ ▷ Use (1).
▷ Note that $R_0^{(1)} = \boldsymbol{I}_3$.
for $t = 1 : t_{\max}$ **do**
$\mathcal{N}^{(i)} \leftarrow \{n \,|\, (n,i) \in \mathcal{E}_i\}$
▷ The set of all neighbors to camera i.
for $n \in \mathcal{N}^{(i)}$ **do**
$\boldsymbol{\omega}^{(n)} \leftarrow \log_{R_{t-1}^{(i)}}(R^{(n,i)} \times R_{t-1}^{(n)})$
end for
$\delta^{(i)} \leftarrow \dfrac{\sum_{n\in\mathcal{N}^{(i)}} \boldsymbol{\omega}^{(n)}/\|\boldsymbol{\omega}^{(n)}\|}{\sum_{n\in\mathcal{N}^{(i)}} 1/\|\boldsymbol{\omega}^{(n)}\|}$
$R_t^{(i)} \leftarrow \exp(\delta^{(i)}) \times R_{t-1}^{(i)}$
end for
return $Rotm2Eul(R_{t_{\max}}^{(i)})$ ▷ Use (2).
end function

and accordingly, the Euler angles change at the rate dictated by their velocities, perturbed by noise:

$$\begin{aligned}\alpha_k^{(i)} &= \alpha_{k-1}^{(i)} + \Delta t\, \dot{\alpha}_{k-1}^{(i)} + e_\alpha \\ \beta_k^{(i)} &= \beta_{k-1}^{(i)} + \Delta t\, \dot{\beta}_{k-1}^{(i)} + e_\beta \\ \gamma_k^{(i)} &= \gamma_{k-1}^{(i)} + \Delta t\, \dot{\gamma}_{k-1}^{(i)} + e_\gamma \end{aligned} \tag{12}$$

where Δt is the sampling time (time interval from $k-1$ to k). Note that the noise terms are samples of i.i.d zero-mean Gaussian distributions with their standard deviations denoted in a similar way, i.e. $\sigma_{\dot\alpha}$, $\sigma_{\dot\beta}$, $\sigma_{\dot\gamma}$, σ_α, σ_β, and σ_γ, respectively.

In the state space, equations (11) and (12) can be written as:

$$x_k^{(i)} = A x_{k-1}^{(i)} + e \tag{13}$$

where

$$A = \begin{bmatrix} I_3 & \Delta t\, I_3 \\ \mathbf{0}_{3\times3} & I_3 \end{bmatrix} \tag{14}$$

$$e \sim \mathcal{N}(\mathbf{0}_6, Q) \tag{15}$$

$$Q = \text{diag}\left(\sigma_\alpha^2, \sigma_\beta^2, \sigma_\gamma^2, \sigma_{\dot\alpha}^2, \sigma_{\dot\beta}^2, \sigma_{\dot\gamma}^2\right). \tag{16}$$

The above model is usually called the state-transition model or *motion model*.

The measurement vector is comprised of the true Euler angles corrupted by measurement noise:

$$z_k^{(i)} = C x_k^{(i)} + e_z \tag{17}$$

where

$$C = \begin{bmatrix} I_3 & \mathbf{0}_{3\times3} \end{bmatrix} \tag{18}$$

and e_z is a sample of a zero-mean Gaussian distribution with the covariance

$$Q_z = \text{diag}\left(\sigma_{z_\alpha}^2, \sigma_{z_\beta}^2, \sigma_{z_\gamma}^2\right) \tag{19}$$

i.e.

$$e_z \sim \mathcal{N}(\mathbf{0}_3, Q_z). \tag{20}$$

Later, we will further elaborate on how the measurements are computed from the actual measurements, i.e. the relative rotations between cameras which are in turn computed by image processing algorithms using overlapping features between camera images.

2) Filter Implementation for Temporal MRA: Our implementation is based on using *particle filters* that are running locally at each camera node in the distributed camera network. The particle filter is a special form of a Bayesian filter, propagating the camera state density through time. Assume that at time k, the density of the state of camera i is approximated by $J^{(i)}$ particles denoted by $\{x_{k,j}^{(i)}\}_{j=1}^{J^{(i)}}$, with each particle associated with a weight denoted by $\{w_{k,j}^{(i)}\}_{j=1}^{J^{(i)}}$, i.e.

$$p_{x_k}^{(i)}(x) \approxeq \sum_{j=1}^{J^{(i)}} w_{k,j}^{(i)}\, \delta\left(x - x_{k,j}^{(i)}\right). \tag{21}$$

Having the particles and weights, the Expected A Posteriori (EAP) estimate of the camera state can be calculated as the weighted average of the particles:

$$\hat{x}_{x_k}^{(i)} = \mathbb{E}_{p_{x_k}^{(i)}}[X] = \sum_{j=1}^{J^{(i)}} w_{k,j}^{(i)}\, x_{k,j}^{(i)}. \tag{22}$$

The filter is comprised of two steps, prediction and update, which run iteratively. In the prediction step, at each time k, the particles from time $k-1$ are propagated to time k according to the motion model. Indeed, for camera i, each particle $x_{k-1,j}^{(i)}$ randomly moves to a new location in the state space, $x_{k|k-1,j}^{(i)} = A\, x_{k-1,j}^{(i)} + e$ where e is a sample of the noise distribution $\mathcal{N}(\mathbf{0}_6, Q)$. The particle weights remain unchanged in the prediction step.

The update step makes use of measurements to update the particle weights according to Bayes' rule. Consequently, those particles that lead to a larger likelihood for the acquired measurement, are given larger (importance) weights. Let us assume that the distributed camera network and relative camera rotations at time k are given in the form of the graph $\mathcal{G}_k = \{\mathcal{V}_k, \mathcal{E}_k\}$. Using the minimum spanning tree (MST) algorithm, we first acquire an initial estimate of each camera orientation, denoted by $\tilde{z}_k^{(i)}$.

A basic overview of the MST algorithm follows; first the most connected node i.e. with the most neighbors, is chosen to be the root node R^{i_0} and is set to have zero rotation. Then, spanning out in a breadth-first fashion from the root node, the other nodes' orientations are estimated using the relation $R^{(n)} = R^{(i,n)} R^{(i)}$ and converted to Euler angles using equation (2). There are

multiple different algorithms for computing the MST of a graph. In this work, Prim's algorithm was used [30]. In this application, edge weights are set to the magnitude of the relative rotation between the two nodes, $|R^{(n,i)}|$.

To allow for the MST algorithm to be used in a distributed network, Algorithm 2 first calculates the weights of the adjacent edges in each node, then uses a message passing algorithm to broadcast the edges and edge weights to each node in the network. This will repeat for a number of iterations, where in each iteration the nodes broadcast the edges and edge weights that they received in the previous time step. Finally, each node estimates its own initial orientation using the MST algorithm. To ensure that each node can calculate the minimum spanning tree from the root node to themselves, the number of messages must be at least as long as the number of edges comprising the radius of the network, which is a task-dependent variable.

We then pass the estimates returned by the MST algorithm, as initial $\boldsymbol{\theta}_0^{(i)}$'s, to the Weiszfeld MRA estimation method presented in Algorithm 1, along with the relative rotations (edges $\mathcal{E}$ of the graph $\mathcal{G}$). The resulting Euler angles form the measurement vectors $z_k^{(i)}$.

Given the measurements $\{z_k^{(i)}\}_{i=2}^N$, noting that the first camera is the reference, for each camera, all the particle weights are re-weighted in proportion to the likelihood of the measurements given the *predicted* particle state. To be precise, each weight $w_{k-1,j}^{(i)}$ is first turned into

$$w'^{(i)}_{k,j} = w^{(i)}_{k-1,j}\, \mathcal{N}\left(z_k^{(i)}; C\, x^{(i)}_{k|k-1,j}, Q_z\right)$$

where $\mathcal{N}(\cdot;\mu,Q)$ denotes the multi-variate Gaussian density function with μ and Q as its mean vector and covariance matrix. Then, the weights are normalized to sum to 1,

$$w^{(i)}_{k,j} = w'^{(i)}_{k,j} \Big/ \sum_{j=1}^{J^{(i)}} w'^{(i)}_{k,j}.$$

The last step involves resampling of the particles to avoid particle death. The resulting samples have equal weights of $1/J^{(i)}$. Algorithm 3 shows the pseudocode of the operations involved in the prediction step.

Algorithm 2: Distributed Message Passing Minimum Spanning Tree Algorithm Within the Single Node Camera i.

procedure DMST(i, d)
▷ i: Specific camera the algorithm is running in.
▷ d: No. of iterative repetitions.
$\mathcal{W}^{(i)} \leftarrow \emptyset$ ▷Initialization
▷ $\mathcal{W}^{(i)}$: set of weights (rotations) corresponding to all edges in the graph whose weights will be communicated to node i.
▷ Note: $\mathcal{W}^{(i)}$ is comprised of rotations such as $R^{(\ell,i)}$ which contain the corresponding edge infor-mation, i.e. (ℓ, i) as well.
$I \leftarrow$Acquire() ; ▷ Image acquisition occurs.
Transmit() ; ▷Transmits image and self-label to all receiving cameras.
$\{(\ell, I_i^{(\ell)})\}_{\ell\in\mathbb{L}_i} \leftarrow$Receive_Images() ;
▷ $\mathbb{L}_i$: set of labels of neighboring cameras.
▷ $I_i^{(\ell)}$: image received from camera ℓ.
for $\ell \in \mathbb{L}_i$ **do**
$R^{(\ell,i)} \leftarrow$Rel_Rot$I, I_i^{(\ell)}$;
▷Computes relative rotation by image comparison.
$\mathcal{W}^{(i)} \leftarrow \mathcal{W}^{(i)} \cup \{R^{(\ell,i)}\}$
end for
for $t = 1 : d$ **do**
Transmit_Rotations($\mathcal{W}^{(i)}$);
▷ Transmits all relative rotations, calculated and received, to all receiving cameras.
$\{\mathcal{W}_\ell^{(i)}\}_{\ell\in\mathbb{L}_i} \leftarrow$Receive_Rotations() ;
▷ Receives from neighboring cameras all the rotations collected by them so far.
▷$\mathcal{W}_\ell^{(i)}$: all relative rotations received from neighboring camera ℓ.
$\mathcal{W}^{(i)} \leftarrow \{\cup_{\ell\in\mathbb{L}_i}\mathcal{W}_\ell^{(i)}\} \cup \mathcal{W}^{(i)}$
▷Note that set union removes all repeated information.
end for
$\{\tilde{R}^{(n)}\}_{n\in\mathcal{N}^{(i)}} \leftarrow$MST($\mathcal{W}^{(i)}$)
▷ Prim's MST algorithm with $R^{(1)}$ as root node.
▷ Note that $\mathcal{W}^{(i)}$ includes edge information.
▷ $\mathcal{N}^{(i)}$ comprises all cameras in the graph formed around camera i after d iterations.
return $\{\tilde{R}^{(n)}\}_{n\in\mathcal{N}^{(i)}}$
end procedure

V. Experiment and Results

A. Datasets

Currently, there is no dataset for MRA that considers temporal information. Therefore, synthetic datasets were created and used to test various models. The code used to generate the datasets is a modified version of the code used in Chaterjee & Govindu [16], which was obtained from Computer Vision Lab [31]. The modifications meant that rather than generating independent data points at each time step, the camera rotations followed the motion model outlined in (13)–(15), where each camera has a random axis of rotation, a random starting orientation $\boldsymbol{\theta}$, and the magnitude of the angular change in orientation $\|\dot{\boldsymbol{\theta}}\|$ is sampled from a Gamma distribution $\Gamma(\alpha = 2.5, \beta = 2)$. This distribution was chosen over more conventional distributions such as the normal distribution to have a peak at 5 degrees, while limiting the possible angles to 0 degrees and having a long right tail to allow the potential for larger rotation angles. The probability density function used to generate camera rotations is shown in Fig. 2. Additionally it is assumed that the change in a camera's orientation from one time step to the next will not be very large so a threshold is chosen at 45°, so that generated angles above the thresholds are resampled.

A further modification allows the possibility of relaxing the requirement for all nodes to have measurements. If this setting is selected, a random selection of nodes have their measurements

Algorithm 3: One Recursion of the Particle Filter Temporal MRA Propagating From $k-1$ to k, Within Camera i.

function PF-MRA($\boldsymbol{x}_{k-1}^{(i)}, A, C, Q, Q_z, d$)
▷ Note that $\boldsymbol{x}_{k-1}^{(i)} = \{x_{k-1,j}^{(i)}\}_{j=1:J^{(i)}}$.
▷Particle weights for camera i are all equal to $\frac{1}{J^{(i)}}$.
$\{\tilde{R}_k^{(n)}\}_{n\in\mathcal{N}^{(i)}} \leftarrow$DMST (i,d) ▷ See Algorithm 2.
$\{\tilde{z}_k^{(n)}\}_{n\in\mathcal{N}^{(i)}} \leftarrow$Rot2Eul$(\{\tilde{R}_k^{(n)}\}_{n\in\mathcal{N}^{(i)}})$
▷ Converts rotation matrix estimates to initial Euler angle estimates.
$z_k^{(i)} \leftarrow$L1W_MRA$\left(\{\tilde{z}_k^{(n)}\}_{n\in\mathcal{N}^{(i)}}, \mathcal{N}^{(i)}, \{R_k^{(n,i)}\}_{n\in\mathcal{N}^{(i)}}, t_{\max}\right)$
▷ See Algorithm 1.
▷ Run the L1 Weiszfeld MRA algorithm on initial Euler angle estimates to acquire Euler angle measurements.
▷Prediction Step
$J^{(i)} \leftarrow |\boldsymbol{x}_{k-1}^{(i)}|$ ▷$|\cdot|$ means the number of elements.
for $j = 1 : J^{(i)}$ **do**
$e \sim \mathcal{N}(\mathbf{0}, Q)$
$x_{k|k-1,j}^{(i)} \leftarrow A x_{k-1,j}^{(i)} + e$
end for
▷ Update Step
for $j = 1 : J^{(i)}$ **do**
$w'^{(i)}_{k,j} \leftarrow \mathcal{N}(z_k^{(i)}; C\, x_{k|k-1,j}^{(i)}, Q_z)$
▷ Since $\forall j$; $w_{k-1,j}^{(i)} = \frac{1}{J^{(i)}}$ and due to normalization in the next step, they are omitted here.
end for
for $j = 1 : J^{(i)}$ **do**
$w_{k,j}^{(i)} \leftarrow w'^{(i)}_{k,j} \Big/ \sum_{j=1}^{J^{(i)}} w'^{(i)}_{k,j}$
▷Normalize the updated particle weights.
end for
▷EAP Estimation and resampling step
$\hat{x}_k^{(i)} \leftarrow \sum_{j=1}^{J^{(i)}} w_{k,j}^{(i)}\, x_{k|k-1,j}^{(i)}$
▷ Note that particles have not changed through update step yet.
▷ Resample the predicted particles according to their updated weights into new $J^{(i)}$ particles.
$\boldsymbol{x}_k^{(i)} \leftarrow Resample(\{\{(x_{k|k-1,j}^{(i)}, w_{k,j}^{(i)})\}_{j=1}^{J^{(i)}}, J^{(i)})$
return $\hat{x}_k^{(i)}, \{\boldsymbol{x}_k^{(i)}\}$
end function

removed from the datasets. This simulates the realistic scenario where a node is disconnected from the graph and cannot generate measurements for a time step for some reason (e.g. weak broadcast signal, signal interference, or running out of battery). To simulate outlier measurements that have been corrupted due to an error in the relative rotation measurement process, several measurements are also replaced with random rotations.

Fig. 2 shows that the majority of rotations are under 10°. This suggests that if a camera n is a neighbor of camera i in time step $k-1$ and $R_{k-1,k-1}^{(n,i)}$ exists, then it is likely that $R_{k-1,k}^{(n,i)}$ also exists. While it is likely, it is not guaranteed that the temporal relative rotation will exist. To reflect this in the synthetic datasets, only a specified percentage of the relative rotations that existed in time step $k-1$ are included as existing temporal relative rotations in time step k.

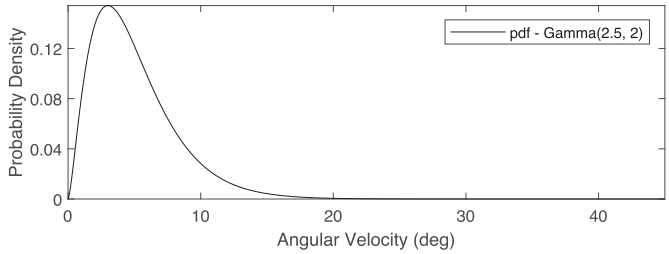


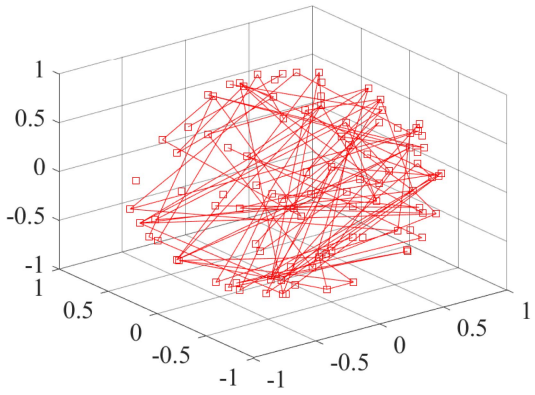
Fig. 2. Hypothesized probability distribution function used to generate the angle of rotation that each camera rotates by each time step.

The main parameters used for generating the datasets used and their values are listed and briefly explained below.

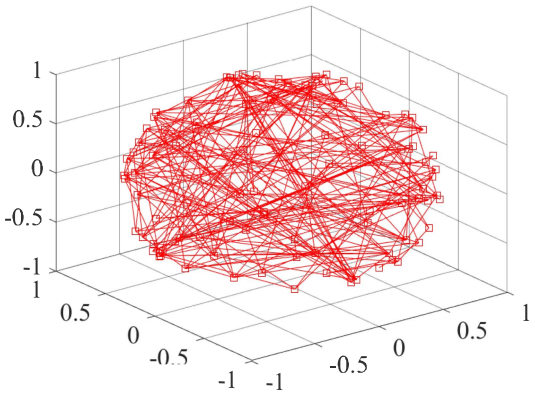
Three collections of datasets were generated, one with 100 cameras and no dropout, another with 50 cameras, and no dropout and one with 50 cameras and node dropout, with each dataset comprising 100 frames. We define two terms to aid in describing how densely connected the camera networks in the datasets are, the first being *spatial density*. Spatial density is the percentage of all possible connections in the camera graph for a single time step that have associated measurements. A fully connected graph of N cameras has $N(N-1)/2$ connections. If the spatial density value is 1, then all possible relative rotations are 'measured' and included in the dataset; if it is 0.5, then only half (rounded up) of the rotations would be 'measured'. For the datasets with 100 cameras, a spatial density of 0.1 was used; for the datasets with 50 cameras, a spatial density of 0.2 was used.

A visualization of how the spatial density value affects the topography of a network of rotations is presented in Fig. 3. For each scenario a number of camera pairs, with overlaps in their fields of view that are large enough to return accurate measurements for relative orientation between those two cameras, are selected based on the spatial density. For each camera, we apply its absolute rotation to the hypothesized location of (1,0,0), and show the rotated point by red small squares. For those pairs of cameras where relative rotation measurements are available, the line connecting the two tiny squares are indicative of the presence of relative rotations.

Temporal density is defined as the percentage of the relative rotation measurements from time step $k-1$ that are recalculated as temporal relative rotations between time steps $k-1$ and k. It is worth noting that a higher density of measurements, e.g. Fig. 3(b) compared to Fig. 3(a), results in multiple measurements involving each individual camera. This is further pronounced with the inclusion of temporal relative rotation measurements. After MRA the measurement error is expected to be drastically reduced, with a higher number of measurements involving a particular camera expected to further reduce the measurement error. Note that assuming Gaussian distribution for



(a) Density = 0.02



(b) Density = 0.05

Fig. 3. Visualization of the effect of spatial density on the topography of the network of rotations from a single frame in a dataset with 100 cameras. Each camera's orientation is represented by a square, positioned according to that camera's rotation matrix applied to a point at coordinates (1, 0, 0). The existing relative rotation measurements are indicated by the lines between two cameras.

measurement noise, with power of σ_n^2, when m such measurements are averaged, the result will suffer from a much weaker noise with a power of $\frac{1}{m}\sigma_n^2$.

For all datasets, a temporal density of 0.5 was used. The number of outliers is the percentage of relative rotation measurements that are simulated as outliers and set to a random rotation. This value varied from 0% to 4%, in 1% increments for all datasets, to measure and compare the models with different noise levels. For the dataset with node dropout, at each time step a random number from a uniform distribution from 0 to 1 is sampled for each camera. If the sampled value is less than 0.1 then that camera's measurements are discarded for that time step, resulting in an average of 10% of cameras being disconnected from the graph at each time step. Additionally, the noise parameters were set to $Q = \text{diag}(0.001, 0.001, 0.001, 0.0003, 0.0003, 0.0003)$ and $Q_z = \text{diag}(0.001, 0.001, 0.001)$ to apply an average of 1.5° of noise for the 100 camera datasets, and $Q = \text{diag}(0.003, 0.003, 0.003, 0.0008, 0.0008, 0.0008)$ and $Q_z = \text{diag}(0.003, 0.003, 0.003)$ to apply an average of 2.5° of noise for the 50 camera datasets. The number of particles representing each camera i is kept at $J^{(i)} = 1000$ at all times.

B. Models

The new approaches are tested and compared to the L1 Weiszfeld MRA algorithm [14]. The code for the base L1 Weiszfeld algorithm was obtained from usstdqq [32] and was modified, alongside the dataset generation code, to incorporate the changes listed above to include temporal information. For the baseline method, at each time step each camera's orientation is initialized using Algorithm 2, followed by the standard L1 Weiszfeld MRA algorithm. Each time step's data is treated independently.

For the first proposed method, using temporal relative rotations, a dataset was generated that included the temporal relative rotations between neighboring cameras from time step $k-1$ to time step k. The index for camera i in time step $k-1$ is set to $N+i$, to ensure each camera and time step combination has a discrete index. The absolute rotation estimates for the first N indexed cameras are taken as the output. Additionally, if a camera's orientation could not be initialized with the MST, then that camera's estimated orientation from the previous time step is used instead.

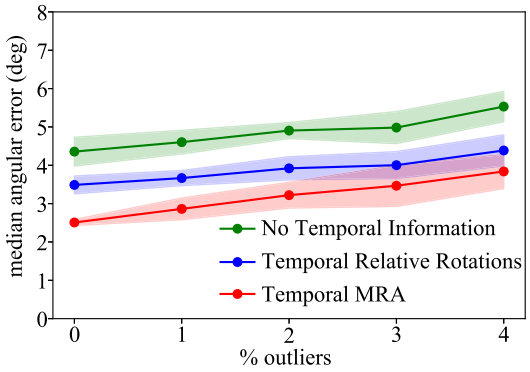
For the second proposed method, using a particle filter with MRA, the baseline method was used for the first time step and Algorithm 3 was used for the remaining time steps.

C. Comparison

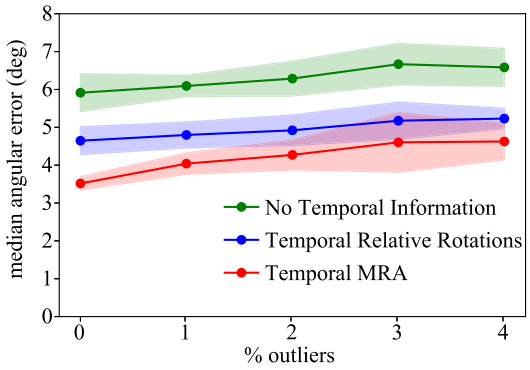
Fig. 4(a) shows estimation errors, demonstrating that an improvement of $20 \pm 0.4\%$ reduction in angular error is achieved when temporal relative rotations are appended to the set of measurements. Fig. 4(b) suggests a slightly larger improvement, with a $21.5 \pm 1\%$ reduction in angular error. This improvement is increased again in Fig. 4(c), with a $23.5 \pm 2.3\%$ reduction in angular error. All figures also demonstrate that there is a significant improvement over the baseline algorithm when using the proposed temporal MRA algorithm with a particle filter, with a $36.5 \pm 6\%$, $35.2 \pm 5.4\%$ and $37.9 \pm 9.6\%$ reduction respectively, which is also an improvement over the temporal relative rotation model.

The relatively greater variance in the scenario in Fig. 4(b), with higher noise, over Fig. 4(a), suggests that the variance in accuracy of each individual time step correlated with the noise level. This is due to a scenario with larger measurement noise resulting in individual measurements' accuracy becoming less reliable. Similarly, the relatively greater variance in the scenario in Fig. 4(c), with node dropout, over the other scenarios without dropout suggests that the variance in accuracy of each individual time step is negatively correlated with the presence of disconnected nodes. Additionally, in all scenarios the increase in the number of inaccurate outlier measurements increases both the error and the variance in the error. Compared to the baseline method, either increasing the number of available measurements through using temporal relative rotations or a particle filter helps mitigate against measurement noise and outliers, improving accuracy.

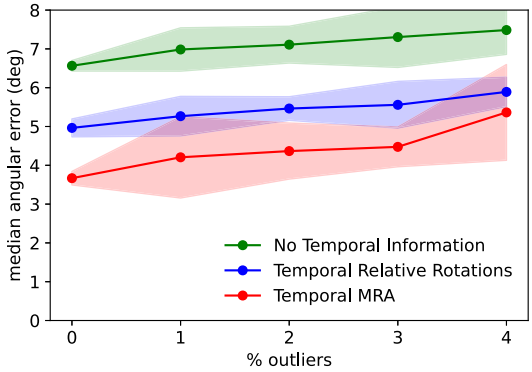
To determine the statistical significance of the results a Monte Carlo Simulation was performed. For each outlier percentage (0%–4%), 20 datasets were generated and were ran through each



(a) 100 cameras, 10% spatial connections, 1.5° average noise, no dropout



(b) 50 cameras, 20% spatial connections, 2.5° average noise, no dropout



(c) 50 cameras, 20% spatial connections, 2.5° average noise, 10% dropout

Fig. 4. Comparison of the median angular error (degrees) of camera orientation estimation between the three methods on the 100 camera datasets. The percentage of outliers was varied from 0% to 4%. Each configuration was run 20 times. The solid line denotes the average median of the runs and the shaded region denotes the range of two standard deviations (95% confidence interval) over the runs.

of the 3 models. The median angular error of the absolute camera orientations for each run, as well as the standard deviation between the runs' median angular errors, was calculated. Paired t-tests was performed between angular errors samples from the baseline model and the temporal relative rotation model, as well as between the temporal relative rotation model and the particle filter model, for each outlier percentage. The obtained p-values are listed in Tables I–III and are all below 0.05, confirming that the improvements are statistically significant, particularly for the particle filter method.

TABLE I
T-TEST P VALUES, 100 CAMERAS WITHOUT DROPOUT

Number of Outlier	Baseline - TRR	TRR - Temporal MRA
0%	9.7395e-11	1.2536e-09
1%	5.8989e-11	3.4565e-09
2%	1.0026e-09	6.0580e-04
3%	4.8593e-09	6.3162e-05
4%	4.0322e-09	2.8643e-03

TABLE II
T-TEST P VALUES, 50 CAMERAS WITHOUT DROPOUT

Number of Outliers	Baseline - TRR	TRR - Temporal MRA
0%	4.5166e-11	1.2802e-09
1%	3.4598e-10	1.8558e-07
2%	5.2882e-08	6.2533e-04
3%	1.5723e-07	2.4586e-04
4%	8.4496e-09	1.6280e-05

TABLE III
T-TEST P VALUES, 50 CAMERAS WITH DROPOUT

Number of Outliers	Baseline - TRR	TRR - Temporal MRA
0%	2.1126e-13	9.0423e-10
1%	3.5807e-09	1.6105e-05
2%	1.9547e-09	3.8985e-06
3%	2.1062e-08	1.3252e-05
4%	4.7180e-09	0.1412e-03

VI. CONCLUSION

The problem of applying multiple rotation averaging to time-series data was introduced and explored. Two solutions were proposed, first an extension of the regular multiple rotation averaging through measuring relative rotations between cameras in different time steps, second by implementing a particle filter based model to exploit temporal data inherent in the problem. Details regarding the construction of appropriate synthetic datasets were also presented. A series of scenarios were developed to compare the proposed solutions to the naïve method of treating every frame as independent multiple rotation averaging problems, and were run as Monte Carlo simulations. These experiments showed a modest improvement in camera orientation accuracy using the temporal relative rotations and a significant improvement in accuracy using the particle filter model. This work can be extended to include translation/position estimation, which combined with multiple rotation averaging would result in temporal multiple transformation averaging for complete motion estimation through time.

REFERENCES

[1] B. Bhanu, C. Ravishankar, A. Roy-Chowdhury, H. Aghajan, and D. Terzopoulos, *Distributed Video Sensor Networks*.Berlin, Germany: Springer, 2011.

[2] W. Jiuqing, C. Xu, B. Shaocong, and L. Li, "Distributed data association in smart camera network via dual decomposition," *Inf. Fusion*, vol. 39, pp. 120–138, 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1566253517302713

[3] X. Fei, L. Li, H. Cao, J. Miao, and R. Yu, "View's dependency and low-rank background-guided compressed sensing for multi-view image joint reconstruction," *IET Image Process.*, vol. 13, no. 12, pp. 2294–2303, 2019, doi: 10.1049/iet-ipr.2019.0295.

[4] A. T. Kamal, J. H. Bappy, J. A. Farrell, and A. K. Roy-Chowdhury, "Distributed multi-target tracking and data association in vision networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, no. 7, pp. 1397–1410, Jul. 2016.

[5] J. H. Ryu, G. Gankhuyag, and K. T. Chong, "Navigation system heading and position accuracy improvement through GPS and INS data fusion," *Hindawi J. Sensors*, vol. 2016, pp. 1–6, 2016.
[6] H. Xu, L. Wang, Y. Zhang, K. Qiu, and S. Shen, "Decentralized visual-inertial-UWB fusion for relative state estimation of aerial swarm," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2020, pp. 8776–8782.
[7] A. K. Burusa, "Visual-inertial odometry for autonomous ground vehicles," Master's thesis, School of Computer Science and Communication (CSC), KTH, Stockholm, Sweden, 2017.
[8] A. Chatterjee and V. M. Govindu, "Robust relative rotation averaging," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 4, pp. 958–972, Apr. 2018.
[9] F. Arrigoni, L. Magri, B. Rossi, P. Fragneto, and A. Fusiello, "Robust absolute rotation estimation via low-rank and sparse matrix decomposition," in *Proc. IEEE 2nd Int. Conf. 3D Vis.*, 2014, pp. 491–498.
[10] B. Triggs, P. F. McLauchlan, R. I. Hartley, and A. W. Fitzgibbon, "Bundle adjustment - a modern synthesis," in *Workshop Vis. Algorithms*, 1999, pp. 298–372.
[11] V. M. Govindu, "Combining two-view constraints for motion estimation," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, 2001, pp. 218–225.
[12] V. Govindu, "Lie-algebraic averaging for globally consistent motion estimation," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, 2004, pp. 684–691.
[13] D. Crandall, A. Owens, N. Snavely, and D. P. Huttenlocher, "SfM with MRFs: Discrete-continuous optimization for large-scale structure from motion," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 12, pp. 2841–2853, Dec. 2013.
[14] R. Hartley, K. Aftab, and J. Trumpf, "L1 rotation averaging using the weiszfeld algorithm," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2011, pp. 3041–3048.
[15] K. Aftab, R. Hartley, and J. Trumpf, "Generalized weiszfeld algorithms for Lq optimization," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 37, no. 4, pp. 728–745, Apr. 2015.
[16] A. Chatterjee and V. M. Govindu, "Efficient and robust large-scale rotation averaging," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2013, pp. 521–528.
[17] F. Arrigoni, B. Rossi, and A. Fusiello, "Spectral synchronization of multiple views in SE(3)," *SIAM J. Imag. Sci.*, vol. 9, pp. 1963–1990, 2016.
[18] F. Arrigoni, B. Rossi, P. Fragneto, and A. Fusiello, "Robust synchronization in SO(3) and SE(3) via low-rank and sparse matrix decomposition," *Comput. Vis. Image Understanding*, vol. 174, pp. 95–113, 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1077314218301528
[19] N. Vaswani, Y. Chi, and T. Bouwmans, "Rethinking PCA for modern data sets: Theory, algorithms, and applications," *Proc. IEEE*, vol. 106, no. 8, pp. 1274–1276, Aug. 2018.
[20] E. J. Candés, X. Li, Y. Ma, and J. Wright, "Robust principal component analysis ?," *J. ACM*, vol. 58, no. 3, Jun. 2011, doi: 10.1145/1970392.1970395.
[21] B. Mehlig, *Mach. Learn. with Neural Netw.* Cambridge, MA, USA: Cambridge Univ. Press, Oct. 2021, doi: 10.1017/9781108860604.
[22] X. Huang, Z. Liang, X. Zhou, Y. Xie, L. J. Guibas, and Q. Huang, "Learning transformation synchronization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019.
[23] P. Purkait, T.-J. Chin, and I. Reid, "Neurora: Neural robust rotation averaging," in *Proc. Eur. Conf. Comput. Vis.*, 2019, pp. 137–154.
[24] L. Yang, H. Li, J. A. Rahim, Z. Cui, and P. Tan, "End-to-end rotation averaging with multi-source propagation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 11774–11783.
[25] S. H. Lee and J. Civera, "HARA: A hierarchical approach for robust rotation averaging," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 15777–15786.
[26] H. Li, Z. Cui, S. Liu, and P. Tan, "RAGO: Recurrent graph optimizer for multiple rotation averaging," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 15787–15796.
[27] K. Kanatani, *Group Theoretic Methods in Image Understanding*. Berlin, Germany: Springer, 1990.
[28] R. I. Hartley, J. Trumpf, Y. Dai, and H. Li, "Rotation averaging," *Int. J. Comput. Vis.*, vol. 103, pp. 267–305, 2012.
[29] X. Liu, H. Shi, X. Hong, H. Chen, D. Tao, and G. Zhao, "3D skeletal gesture recognition via hidden states exploration," *IEEE Trans. Image Process.*, vol. 29, pp. 4583–4597, 2020.
[30] R. C. Prim, "Shortest connection networks and some generalizations," *Bell Syst. Tech. J.*, vol. 36, no. 6, pp. 1389–1401, 1957.
[31] A. Chatterjee and V. M. Govindu, "Efficient and robust large-scale rotation averaging," *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 521–528, [Online]. Available: https://ee.iisc.ac.in/cvlab/research/rotaveraging/
[32] Q. Dai, "Mean rotation in l1 norm by using the weiszfeld algorithm," 2021, [Online]. Available: https://sites.google.com/site/qiqindai1990/codes